

BAB 10

OPERASI FILE

Tujuan

1. Memberikan pemahaman kepada mahasiswa tentang operasi *file* di Java.

Ringkasan Materi

Data yang diolah di dalam program bersifat sementara dan akan hilang saat program ditutup. Untuk menyimpan data yang dibuat oleh program, data tersebut harus disimpan di dalam sebuah *file*. Data tersebut kemudian akan dapat dibaca dan dipindah ke tempat yang lain, dibaca oleh program yang lain, dst.

Setiap *file* disimpan di dalam suatu direktori. Suatu *file* memiliki nama absolut yang berisi nama *drive*, direktori, nama *file* itu sendiri, hingga ekstensinya. Contohnya, "C:\Data\Java\teks.txt" adalah nama absolut dari *file* "teks.txt" yang berada di dalam direktori/path "C:\Data\Java" pada sistem operasi Windows. Sistem operasi yang lain memiliki format penamaan yang berbeda. Contohnya, pada sistem operasi berbasis UNIX, nama absolut dari *file* yang sama adalah "/Data/Java/teks.txt". Perlu diperhatikan pula bahwa pada sistem operasi berbasis UNIX, nama *file* bersifat *case sensitive*, sedangkan pada Windows tidak.

Operasi *file* termasuk di dalam kategori input-output (I/O) sehingga diletakkan pada *package* java.io yang merupakan *package* input-output standar di Java. Java menyediakan sekumpulan *class* baru yang dinamakan Java NIO (new IO) dan diletakkan pada *package* java.nio. *Package-package* tersebut juga menyediakan operasi-operasi input-output untuk jaringan.

Operasi input-output berpotensi untuk memunculkan *error*. Oleh karena itu, terdapat kelas *exception* khusus untuk input-output, yaitu java.io.IOException. *File* merupakan suatu *resource* yang dapat dibuka dan ditutup. Oleh karena itu, kita bisa memanfaatkan fitur *try-with-resource* yang akan akan menutup *resource* secara otomatis setelah operasi terhadap *resource* tersebut selesai. Hal ini penting, terutama pada komputer dengan kapasitas memori yang terbatas dan jika *file* yang digunakan berukuran besar. Untuk kemudahan implementasi, kita bisa menggunakan *library* Apache Commons IO (<https://commons.apache.org/io>) yang menyediakan *method-method* untuk operasi-operasi input-output yang umum.

1. Kelas File

Kelas File adalah kelas yang dapat merepresentasikan *file* dan direktori. Kelas ini dapat digunakan untuk membuat, mendapatkan properti, mengubah nama (*rename*), dan menghapus suatu *file*/direktori. Namun, kelas ini saja tidak dapat digunakan untuk membaca dan menulis ke *file*. Membuat suatu objek kelas File tidak akan secara langsung membuat *file* baru, melainkan hanya sebuah penunjuk/*pointer* ke suatu *file*/direktori saja. Untuk membaca dan menulis ke suatu *file*, dibutuhkan beberapa kelas yang lain. Untuk mengecek apakah suatu objek File merepresentasikan *file* atau direktori, kita dapat menggunakan method *isDirectory()* dan *isFile()*. Tabel 1 menampilkan beberapa *method* yang terdapat pada kelas File.

Tabel 1. Beberapa method yang ada pada kelas File.

No.	Method	Deskripsi
-----	--------	-----------

1	canExecute()	Mengecek apakah suatu <i>file</i> dapat dieksekusi.
2	canRead()	Mengecek apakah suatu <i>file</i> dapat dibaca.

3	canWrite()	Mengecek apakah suatu <i>file</i> dapat ditulis.
4	createNewFile()	Membuat <i>file</i> kosong baru.
5	delete()	Menghapus <i>file</i> /direktori.
6	deleteOnExit()	Menghapus <i>file</i> /direktori setelah eksekusi program berakhir.
7	equals(Object o)	Mengecek kesamaan dengan objek o.
8	exists()	Mengecek keberadaan <i>file</i> /direktori.
10	getAbsolutePath() ()	Mendapatkan nama absolut suatu <i>file</i> /direktori.
11	getFreeSpace()	Mendapatkan <i>free space</i> dari <i>drive</i> /partisi.
12	getName()	Mendapatkan nama <i>file</i> /direktori.
13	getParent()	Mendapatkan nama <i>parent</i> dari <i>file</i> /direktori sebagai <i>string</i> .
14	getParentFile()	Mendapatkan <i>parent</i> dari <i>file</i> /direktori sebagai objek File.
16	getTotalSpace()	Mendapatkan ukuran dari <i>drive</i> /partisi.
17	getUsableSpace() ()	Mendapatkan ukuran yang dapat digunakan dari <i>drive</i> /partisi.
18	isAbsolute()	Mengecek apakah nama <i>file</i> /direktori bersifat absolut.
19	isDirectory()	Mengecek apakah objek berupa direktori.
20	isFile()	Mengecek apakah objek berupa <i>file</i> .
21	isHidden()	Mengecek apakah <i>file</i> /direktori berstatus <i>hidden</i> .
22	lastModified()	Mendapatkan waktu terakhir <i>file</i> /direktori dimodifikasi.
23	length()	Mendapatkan ukuran <i>file</i> /direktori dalam satuan <i>bytes</i> .
24	list()	Mendapatkan daftar <i>file</i> /direktori di dalam suatu direktori.

2 5	<code>listFiles()</code>	Mendapatkan daftar <i>file</i> /direktori di dalam suatu direktori sebagai objek File.
2 6	<code>mkdir()</code>	Membuat direktori baru.
2 7	<code>mkdirs()</code>	Membuat beberapa direktori baru.
2 8	<code>renameTo()</code>	Mengubah nama <i>file</i> /direktori.
2 9	<code>setExecutable()</code>	Mengubah status <i>file</i> menjadi <i>executable</i> .
3 0	<code>setLastModified() ()</code>	Mengubah waktu modifikasi terakhir <i>file</i> /direktori.
3 1	<code>setReadOnly()</code>	Mengubah status <i>file</i> menjadi <i>read only</i> .
3 2	<code>setReadable()</code>	Mengubah status <i>file</i> menjadi <i>readable</i> .
3 3	<code>setWritable()</code>	Mengubah status <i>file</i> menjadi <i>writable</i> .

2. Menulis Teks ke File

Terdapat banyak cara dan *class* di Java untuk membaca isi *file*, antara lain `OutputStream`, `ByteArrayOutputStream`, `FileOutputStream`, `RandomAccessFile`, `PipedOutputStream`, `BufferedOutputStream`, `FilterOutputStream`, `DataOutputStream`, `PrintStream`, `ObjectOutputStream`, `FileWriter`, `PipedWriter`, `BufferedWriter`, `FilterWriter`, `StringWriter`, dan `PrintWriter`, dan `Files` (NIO).

3. Membaca Isi File

Terdapat banyak cara dan *class* pula di Java untuk membaca isi *file*, antara lain `InputStream`, `ByteArrayInputStream`, `FileInputStream`, `RandomAccessFile`, `PipedInputStream`, `BufferedInputStream`, `FilterInputStream`, `DataInputStream`, `ObjectInputStream`, `FileReader`, `PipedReader`, `BufferedReader`, `FilterReader`, `Scanner`, `StringReader`, dan `Files` (NIO).

Sebelum melakukan operasi pembacaan isi *file*, sangat disarankan untuk mengecek apakah *file* yang akan dibaca benar-benar ada dan dapat dibaca. Hal ini penting untuk mencegah munculnya *error* jika *file* yang ingin dibaca ternyata tidak ada atau tidak dapat dibaca. Pengecekan ini dapat dilakukan dengan `method exists()` dan `canRead()`.

4. Mengubah Nama dan Menghapus File

Mengubah nama *file*/direktori dilakukan menggunakan *method* `renameTo()`,

sedangkan menghapus *file*/direktori dilakukan menggunakan *method* `delete()` atau `deleteOnExit()`. Sama halnya dengan membaca *file*, sangat disarankan untuk mengecek keberadaan *file* yang akan diubah namanya atau dihapus. Method `renameTo()` dan `delete()` mengembalikan nilai Boolean yang mengindikasikan apakah operasi berhasil atau tidak. Untuk operasi hapus, jika yang akan dihapus adalah sebuah direktori, maka direktori tersebut harus dalam keadaan kosong. Jika tidak, maka operasi akan gagal.

Pelaksanaan Percobaan

A. Menulis teks ke *file*

Ketikkan kode program di bawah ini dan analisis *output* dari program tersebut!

TulisFile1.java	
1	<code>import java.io.FileWriter;</code>
2	<code>import java.io.IOException;</code>
3	<code>import java.util.Scanner;</code>
4	
5	<code>public class TulisFile {</code>
6	
7	<code> public static void main(String[] args) {</code>
8	<code> var keyboard = new Scanner(System.in);</code>
9	
10	<code> System.out.print("Masukkan teks yang akan disimpan: ");</code>
11	<code> var text = keyboard.nextLine();</code>
12	
13	<code> try (var writer = new FileWriter("test.txt", false)) {</code>
14	<code> writer.write(text);</code>
15	<code> } catch (IOException e) {</code>
16	<code> System.err.println("Gagal menulis ke file"); } </code>
17	<code> }</code>
18	<code> }</code>
19	
20	

TulisFile2.java	
1	<code>import java.io.IOException;</code>
2	<code>import java.nio.file.Files;</code>
3	<code>import java.nio.file.Paths;</code>
4	<code>import java.util.Scanner;</code>
5	
6	<code>public class TulisFile2 {</code>
7	
8	<code> public static void main(String[] args) {</code>
9	<code> var keyboard = new Scanner(System.in);</code>
10	<code> var path = Paths.get("test.txt");</code>
11	

12	
13	System.out.print("Masukkan teks yang akan disimpan: "); var text = keyboard.nextLine();

14	try {
15	Files.writeString(path, text);
16	} catch (IOException e) {
17	System.err.println("Gagal menulis ke file"); }
18	}
19	}
20	
21	

TulisFile3.java	
-----------------	--

1	import java.io.IOException;
2	import java.io.PrintWriter;
3	import java.util.Scanner;
4	
5	public class TulisFile3 {
6	
7	public static void main(String[] args) {
8	var keyboard = new Scanner(System.in);
9	
10	System.out.print("Masukkan teks yang akan disimpan: ");
11	var text = keyboard.nextLine();
12	
13	try (var writer = new PrintWriter("test.txt")) {
14	writer.println(text);
15	} catch (IOException e) {
16	System.err.println("Gagal menulis ke file"); }
17	}
18	}
19	

B. Membaca isi file teks

Ketikkan kode program di bawah ini dan analisis *output* dari program tersebut!

BacaFile1.java

```

1  import java.io.BufferedReader;
2  import java.io.File;
3  import java.io.FileReader;
4  import java.io.IOException;
5
6  public class BacaFile1 {
7
8      public static void main(String[] args) {
9          var filename = "test.txt";
1         var file = new File(filename);
1
2         if (file.exists() && file.canRead()) {
1
2             try (var reader = new FileReader(filename); var buffer
3             = new BufferedReader(reader)) { var line =
1             buffer.readLine();
3
4             while (line != null) {
1                 System.out.println(line);
4                 line = buffer.readLine();
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

2      }
1      } catch (IOException e) {
2          System.err.println("Gagal membaca file"); }
2      } else {
2          System.out.println("File tidak ada atau tidak bisa
3          dibaca");
2      }
4      }
2  }
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

8
2
9

BacaFile2.java

```
1  import java.io.File;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.Scanner;
5
6  public class BacaFile2 {
7
8      public static void main(String[] args) {
9          var filename = "test.txt";
1         var file = new File(filename);
1
2         if (file.exists() && file.canRead()) {
1
2             try (var reader = new FileReader(filename); var
3 scanner = new Scanner(reader)) { while
1 (scanner.hasNextLine()) {
3     System.out.println(scanner.nextLine()); }
1     } catch (IOException e) {
4     System.err.println("Gagal membaca file"); }
1     } else {
5     System.out.println("File tidak ada atau tidak bisa
1 dibaca");
6     }
1     }
7 }
1
2
3
4
5
```

2	
6	

BacaFile3.java	
1	import java.io.IOException;
2	import java.nio.file.Files;
3	import java.nio.file.Paths;
4	import java.util.List;
5	
6	public class BacaFile3 {
7	
8	public static void main(String[] args) {
9	var path = Paths.get("test.txt");
1	var file = path.toFile();
0	
1	if (file.exists() && file.canRead()) {
1	
1	
2	

1	List<String> text = null;
3	
1	try {
4	text = Files.readAllLines(path);
1	} catch (IOException e) {
5	System.err.println("Gagal membaca file"); }
1	
6	for (var line : text) {
1	System.out.println(line);
7	}
1	} else {
8	System.out.println("File tidak ada atau tidak bisa
1	dibaca");
9	}
2	}
0	}
2	
1	
2	
2	
2	
3	
2	
4	
2	

5	
2	
6	
2	
7	
2	
8	

C. Informasi tentang *file*

Ketikkan kode program di bawah ini dan analisis *output* dari program tersebut!

FileInfo.java	
1	import java.io.File;
2	import java.util.Date;
3	
4	public class FileInfo {
5	
6	public static void main(String[] args) {
7	var file = new File("test.txt");
8	
9	System.out.println("File ada? " + file.exists());
10	System.out.printf("Path: %s\n",
	file.getAbsolutePath());
11	System.out.println("Last modified: " + new
	Date(file.lastModified()));
12	System.out.printf("Ukuran file: %d bytes\n",
	file.length());
13	System.out.println("File bisa dibaca? " +
	file.canRead());
14	System.out.println("File bisa ditulis? " +
	file.canWrite());
15	System.out.println("Apakah sebuah direktori? " +
	file.isDirectory());
16	System.out.println("Apakah sebuah file? " +
	file.isFile());
17	System.out.println("Apakah hidden? " +
18	file.isHidden());
	}
19	}
20	

D. Mengubah Nama *File*

Ketikkan kode program di bawah ini dan analisis *output* dari program tersebut!

Rename.java

1	import java.io.File;
2	
3	public class Rename {
4	
5	public static void main(String[] args) {
6	var file = new File("test.txt");
7	var fileBaru = new File("test-baru.txt");
8	
9	if (file.exists() && !fileBaru.exists()) {
10	var result = file.renameTo(fileBaru);
11	
12	System.out.println((result ? "Berhasil" : "Gagal") + "
	mengubah nama file");
13	} else {
14	System.out.println("File tidak ada atau file dengan nama
	yang baru sudah ada");
15	}
16	}
17	}

E. Menghapus File

Ketikkan kode program di bawah ini dan analisis *output* dari program tersebut!

Delete.java	
1	import java.io.File;
2	
3	public class Delete {
4	
5	public static void main(String[] args) {
6	var file = new File("test-baru.txt");
7	
8	if (file.exists()) {
9	var result = file.delete();
10	
11	System.out.println((result ? "Berhasil" : "Gagal") + "
12	menghapus file");
	} else {
13	System.out.println("File tidak ada"); }
14	}
15	}
16	

Tugas Praktikum

1. Jalankan kode TulisFile1.java beberapa kali dan amati yang terjadi pada file yang ditulis. Kemudian, Pada baris 13, ubah parameter *false* menjadi *true*. Kemudian jalankan kode tersebut beberapa kali dan amati yang terjadi pada *file* yang ditulis.

2. Buat kode program untuk mendapatkan ukuran *file* dalam satuan KB jika ukuran *file* tersebut < 1 MB dan dalam satuan MB jika ukuran *file* tersebut >= 1 MB.
3. Buat kode program untuk menampilkan nama dari semua *file* yang ada di dalam suatu direktori. Petunjuk: gunakan perulangan dan *method* `list()` atau `listFiles()`.
4. Buat kode program untuk menghapus suatu direktori beserta semua *file* yang ada di dalamnya. Asumsi: di dalam direktori tersebut, hanya ada *file-file* saja, tidak ada subdirektori.
5. Apakah yang salah dengan *statement* berikut? Berikan penjelasan.

```
var file = new File("C:\Data\Java\teks.txt");
```
6. Apa yang akan terjadi jika kita mencoba untuk membaca isi dari suatu *file* tetapi *file* tersebut tidak ada dan kita tidak melakukan pengecekan lebih dulu?

Jawab:

1. Bila pada parameter diubah menjadi "true" tulisan akan ditambahkan ke file tanpa menghapus tulisan yang lama sebaliknya jika false maka tulisan yang lama akan hilang jika user menulis tulisan baru pada file
5. Backslash dalam penulisan alamat file yang benar adalah seperti ini "\\", contoh:
"D:\\java\\pembantugas9"
6. Jika kita mencoba untuk membaca file yang tidak ada tanpa pengecekan terlebih dahulu, akan muncul exception `FileNotFoundException`. Untuk mencegah hal ini, sebaiknya lakukan pengecekan apakah file tersebut ada menggunakan `exists()` sebelum mencoba membacanya.